

Engineering Lessons Learned and Design Recommendations

The following lessons supplement the Project A design changes documented in the Error Sheet, which should also be reviewed before further development.

1. Adjustment of MPU6050_tockn Library

The MPU module used in our system did not behave exactly like a standard MPU6050. During initial testing with the Adafruit MPU6050 library, the sensor returned a non-standard WHO_AM_I identification value instead of the expected MPU6050 value, indicating that the module was most likely a compatible or non-original MPU variant. Because of this, the standard Adafruit library did not initialize the sensor reliably, so we switched to / modified the mpu6050_tockn library in order to communicate with the module despite its non-standard identification behavior.

After communication was established, the sensor data recorded in the CSV file showed saturation during fast spinning. This indicated that the default measurement ranges used by the library were too narrow for the spinning-top system. Therefore, the gyroscope full-scale range was changed from ± 500 deg/sec to ± 2000 deg/sec by writing 0x18 to the GYRO_CONFIG register instead of 0x08. Since this configuration changes the gyro sensitivity, the gyro scale factor was also updated from 65.5 to 16.4 LSB/(deg/sec). This correction was applied both in the gyro offset calibration function and in the update function, so that the calculated offsets, angular velocity values, and integrated gyro angles would all use the same physical scale.

In addition, the accelerometer full-scale range was changed from $\pm 2g$ to $\pm 4g$ by writing 0x08 to the ACCEL_CONFIG register instead of 0x00. Accordingly, the accelerometer scale factor in the update function was changed from 16384 to 8192 LSB/g. This ensures that the raw acceleration values are converted according to the actual configured accelerometer range, and that the acceleration-based angle estimates are calculated using the correct physical scale.

Overall, these changes were necessary because the default library configuration did not match the behavior and dynamic range required by our hardware. Updating both the register settings and the corresponding conversion factors prevented saturated or incorrectly scaled measurements and made the MPU data more suitable for the spinning-top system.

```
void MPU6050::begin(){
  writeMPU6050(MPU6050_SMP_LRT_DIV, 0x00);
  writeMPU6050(MPU6050_CONFIG, 0x00);

  //writeMPU6050(MPU6050_GYRO_CONFIG, 0x08); // ±500 deg/sec
  writeMPU6050(MPU6050_GYRO_CONFIG, 0x18); // ±2000 deg/sec

  // g = gravity acceleration
  //writeMPU6050(MPU6050_ACCEL_CONFIG, 0x00); // ±2[g]
  writeMPU6050(MPU6050_ACCEL_CONFIG, 0x08); // ±4[g]

  writeMPU6050(MPU6050_PWR_MGMT_1, 0x01);
  this->update();
  angleGyroX = 0;
  angleGyroY = 0;
  angleX = this->getAccAngleX();
  angleY = this->getAccAngleY();
  preInterval = millis();
}

void MPU6050::calcGyroOffsets(bool console, uint16_t delayBefore, uint16_t delayAfter){
  float x = 0, y = 0, z = 0;
  int16_t rx, ry, rz;

  delay(delayBefore);
  if(console){
    Serial.println();
    Serial.println("=====");
    Serial.println("Calculating gyro offsets");
    Serial.println("DO NOT MOVE MPU6050");
  }
  for(int i = 0; i < 3000; i++){
    if(console && i % 1000 == 0){
      Serial.print(".");
    }
    wire->beginTransaction(MPU6050_ADDR);
    wire->write(0x43);
    wire->endTransmission(false);
    wire->requestFrom((int)MPU6050_ADDR, 6);

    rx = wire->read() << 8 | wire->read();
    ry = wire->read() << 8 | wire->read();
    rz = wire->read() << 8 | wire->read();

    //x += ((float)rx) / 65.5;
    //y += ((float)ry) / 65.5;
    //z += ((float)rz) / 65.5;
    x += ((float)rx) / 16.4;
    y += ((float)ry) / 16.4;
    z += ((float)rz) / 16.4;
  }

  gyroXOffset = x / 3000;
  gyroYOffset = y / 3000;
  gyroZOffset = z / 3000;

  if(console){
    Serial.println();
    Serial.println("Done!");
    Serial.print("X : ");Serial.println(gyroXOffset);
    Serial.print("Y : ");Serial.println(gyroYOffset);
    Serial.print("Z : ");Serial.println(gyroZOffset);
    Serial.println("Program will start after 3 seconds");
    Serial.print("=====");
    delay(delayAfter);
  }
}
```

```
void MPU6050::update(){
  wire->beginTransaction(MPU6050_ADDR);
  wire->write(0x3B);
  wire->endTransmission(false);
  wire->requestFrom((int)MPU6050_ADDR, 14);

  rawAccX = wire->read() << 8 | wire->read();
  rawAccY = wire->read() << 8 | wire->read();
  rawAccZ = wire->read() << 8 | wire->read();
  rawTemp = wire->read() << 8 | wire->read();
  rawGyroX = wire->read() << 8 | wire->read();
  rawGyroY = wire->read() << 8 | wire->read();
  rawGyroZ = wire->read() << 8 | wire->read();

  temp = (rawTemp + 12412.0) / 340.0;

  //accX = ((float)rawAccX) / 16384.0;
  //accY = ((float)rawAccY) / 16384.0;
  //accZ = ((float)rawAccZ) / 16384.0;
  accX = ((float)rawAccX) / 8192.0;
  accY = ((float)rawAccY) / 8192.0;
  accZ = ((float)rawAccZ) / 8192.0;

  angleAccX = atan2(accY, accZ + abs(accX)) * 360 / 2.0 / PI;
  angleAccY = atan2(accX, accZ + abs(accY)) * 360 / -2.0 / PI;

  //gyroX = ((float)rawGyroX) / 65.5;
  //gyroY = ((float)rawGyroY) / 65.5;
  //gyroZ = ((float)rawGyroZ) / 65.5;
  gyroX = ((float)rawGyroX) / 16.4; // 16.4 LSB/(deg/sec)
  gyroY = ((float)rawGyroY) / 16.4; // 16.4 LSB/(deg/sec)
  gyroZ = ((float)rawGyroZ) / 16.4; // 16.4 LSB/(deg/sec)

  gyroX -= gyroXOffset;
  gyroY -= gyroYOffset;
  gyroZ -= gyroZOffset;

  interval = (millis() - preInterval) * 0.001;

  angleGyroX += gyroX * interval;
  angleGyroY += gyroY * interval;
  angleGyroZ += gyroZ * interval;

  angleX = (gyroCoef * (angleX + gyroX * interval)) + (accCoef * angleAccX);
  angleY = (gyroCoef * (angleY + gyroY * interval)) + (accCoef * angleAccY);
  angleZ = angleGyroZ;

  preInterval = millis();
}
```

2. Hammer Task Timing and Power Stress

Reducing the hammer task period below approximately **50 [ms]** may significantly increase the actuator switching rate and the average current demand. During testing, aggressive hammer timing was associated with excessive stress on the ESP32 power path and may lead to regulator overheating or failure. Any reduction in the task period should therefore be performed gradually while monitoring the supply voltage, current consumption, and regulator temperature.

3. Mechanical Overspeed During Drill-Assisted Spin-Up

The spinning top should not be accelerated excessively using the drill.

High rotational speeds generate large centrifugal forces that may tear coil wires, damage solder joints, or detach mechanically unsupported components.

Spin-up should be gradual, and the maximum safe rotational speed should be determined experimentally before full-system operation.

4. Lithium-Ion Cell Interconnection

Lithium-ion cells should not be connected in series by soldering directly onto the cell terminals.

Prolonged heating may damage the cells, degrade their internal structure, or create a safety hazard.

Series connections should be implemented using **spot-welded nickel strips**, pre-tabbed cells, or suitable battery holders. Removable high-current connectors, such as **T-type connectors**, are recommended for connecting the completed battery pack to the system.

5. 3S Charger and Cell-Voltage Matching

Before connecting cells to the 3S charging or protection module, the individual cell voltages must be checked and brought to a similar level.

Connecting significantly unbalanced cells may cause excessive balancing current, incorrect charger operation, or permanent damage to the module.

The cells should be matched, verified individually, and connected according to the required BMS/charger wiring sequence.

6. VL-Based Angle Correction Limitations

The VL53L0X sensors were found to be unreliable as a continuous angular reference.

They operate at a lower sampling rate than the MPU6050 (**20 [ms] compared with 5 [ms]**) - and frequently produce invalid measurements that are replaced by fallback values. As a result, a valid VL correction is not available for every MPU angle update.

The software supports a fused-angle calculation in which the MPU-derived angle is corrected only when a fresh and valid VL measurement is available. When no valid VL measurement is present, the fused estimate continues using the MPU-based angle. Because the VL corrections were not sufficiently consistent, the final POV implementation uses the more reliable MPU-derived angle.

7. MPU6050 Startup Calibration

The MPU6050 is calibrated during system startup.

The spinning top must remain completely stationary throughout this stage.

Motion, vibration, or manual handling during calibration may introduce an incorrect gyroscope offset, causing angular-velocity errors and increased angle drift during operation.

A visible calibration-state indication or delayed system activation is recommended in future versions.

8. Accumulated Gyroscope Drift

The MPU-based angle is obtained by integrating angular velocity and therefore remains sensitive to accumulated gyroscope bias and drift. A more reliable absolute angular reference should be added in a future design to periodically correct the estimated orientation.

The current POV implementation includes an **aging-compensation mechanism** that projects the most recent MPU angle to the actual LED display time: $\theta_{display} = \theta_{MPU} + \omega_z \Delta t_{age}$

This reduces display phase error caused by measurement and processing latency, especially when the rotational speed changes. However, it does not correct accumulated gyroscope drift.

9. Fallback Values Must Not Be Treated as Valid Measurements

Fallback values are useful for maintaining continuous data transmission and preventing software failures, but they must not be interpreted as real sensor measurements. Every sensor value should be accompanied by a validity or freshness indication. Invalid VL measurements should not be used for angle correction, sensor fusion, or physical conclusions.

10. GPIO, UART, and Boot-Pin Conflicts

Pin allocation must be completed before integrating the full system. Several ESP32 pins are shared with UART communication, serial debugging, motor drivers, LEDs, or boot functions. A pin that operates correctly during isolated testing may interfere with another subsystem after integration. Future hardware revisions should avoid bootstrapping pins and reserve dedicated pins for UART, debugging, actuators, and sensor shutdown control.

11. Real-Time Tasks Must Be Isolated from Communication and Logging

High-frequency sensor acquisition and POV timing should not be blocked by WebSocket transmission, CSV writing, file flushing, or Android visualization. These operations should remain in separate FreeRTOS tasks or run at controlled rates. Shared data should be transferred through synchronized snapshots so that the angle, angular velocity, sensor values, and timestamp originate from the same acquisition state.

The use of binary packets, communication throttling, fixed graph buffers, and mutex-protected snapshots reduced latency, jitter, and inconsistent data access in the final system.

12. Preserve Sensor Timestamps

Each measurement should retain its original acquisition timestamp rather than receiving a new timestamp during transmission or logging. This is necessary for comparing sensors with different sampling rates, evaluating data age, applying POV aging compensation, and maintaining consistency between the CSV file and Android visualization.

13. Rebalance the System After Every Hardware Modification

Any change to the battery pack, charger module, wiring, solder volume, PCB-mounted components, or mechanical supports changes the center of mass. The system should therefore be rebalanced after every hardware modification. Dynamic testing should only begin after static balance has been verified using the balancing stand.

14. Verify Mechanical Compatibility Before Final Assembly

Electrical suitability does not guarantee mechanical compatibility. The selected 3S charging module and the revised battery arrangement created enclosure and soldering difficulties and prevented the spinning top from remaining permanently closed. Future versions should verify component dimensions, connector access, wire-routing space, and enclosure clearance before final PCB and mechanical design. A modular multi-layer structure is preferable when the power electronics cannot be integrated into a single closed layer.

15. Perform Integration Incrementally

Sensors, actuators, communication, logging, and power modules should first be validated independently and then added to the system one subsystem at a time. Testing the complete system immediately makes it difficult to distinguish electrical failures, scheduling interference, communication overload, and mechanical instability. Each integration stage should include current measurements, timing verification, sensor-validity checks, and a short mechanical spin test.